

Guía de UI, Pausa, Menú e Inventario (Unity 2D)

Esta guía resume cómo montar la interfaz del juego (menú principal, pausa, botones con hover/click) y un inventario básico reutilizable. Se asume Unity con Input clásico y EventSystem.

1) Estructura de escenas

- MainMenu: pantalla principal con botones Jugar, Seleccionar Nivel (opcional), Salir.
- Level_01 (tu nivel actual): contiene UI in-game (HUD/Inventario) y Panel de Pausa.

2) Menú principal

1. Crea Canvas (Screen Space - Overlay), agrega Panel de fondo.
2. Añade botones (UI > Button) y agrega **UIButtonHover** a cada uno para hover/click visual.
3. Añade **MainMenuController** a un GO vacío y conecta:
 - Botón Jugar →OnClick → MainMenuController.Play()
 - O usa **LoadSceneByName("Level_01")** si cambias el nombre.
 - Botón Salir →OnClick → MainMenuController.Quit()
4. Asegúrate de incluir las escenas en Build Settings.

3) Pausa en nivel (GameManager unificado)

1. En la escena del nivel, asegúrate de tener un **GameManager** (único) con:
 - **pausePanel** asignado (Panel de pausa en Canvas) y **winPanel** si lo usas.
 - Metas opcionales: **targetCoins**, **targetEnemies**.
2. Crea un Panel de Pausa dentro de un Canvas (Screen Space - Overlay):
 - Botones: Reanudar, Reiniciar, Menú.
 - Añade **UIButtonHover** a los botones para feedback visual.
 - Conecta sus OnClick directamente a métodos del HUD (que internamente llaman al GameManager):
 - Reanudar → **HUD.UI_Resume()**
 - Reiniciar → **HUD.UI_RestartLevel()**
 - Menú → **HUD.UI_ExitToMainMenu()**
3. Pulsa ESC para alternar pausa: el **GameManager** captura ESC en **Update()** y llama Pause/Resume.

4) Botones de Unity: cómo funcionan y parámetros

Los **Button** de Unity incluyen transición visual integrada y eventos:

- Transition:
 - Color Tint: define Normal, Highlighted, Pressed, Selected, Disabled. Úsalo si solo necesitas cambiar color/alpha.
 - Sprite Swap: reemplaza el sprite según estado.
 - Animation: anima propiedades con un Animator.
- Interactable: habilita/deshabilita el botón.
- Navigation: navegación por teclado/gamepad (Automatic/Explicit/None).

- Target Graphic: el Graphic (Image/Text) que se colorea en Color Tint.
- OnClick(): lista de callbacks. Aquí conectas métodos públicos (por ejemplo `HUD.UI_Resume()`).

¿Cuándo usar `UIButtonHover`?

- Si necesitas lógica adicional en hover/click (sonido, escala, efectos que no cubre Color Tint) sin crear un Animator.
- Implementa `IPointerEnter/Exit/Down/Up/Click` para reaccionar con precisión (p.ej., reproducir sonido al entrar/salir, escalar levemente el botón).

¿Button nativo o script personalizado?

- Usa SOLO `Button` (recomendado por defecto) cuando:
 - Quieres un OnClick estándar.
 - El feedback visual se resuelve con Transition = Color Tint o Sprite Swap.
 - Requieres navegación por teclado/mandos usando Navigation del Button.
- Usa `Button` + script ligero (interfaces) cuando:
 - Necesitas efectos en hover/press (escala, sonido, tooltips) sin montar un Animator.
 - Quieres “press & hold”, doble click o reglas adicionales antes de permitir OnClick.
- Usa SOLO script (sin `Button`) cuando:
 - Es un control altamente custom (menú radial, drop zones, knobs) donde el patrón de Button no encaja.

Checklist rápido:

- ¿Solo color/alpha por estado? → Button (Color Tint)
- ¿Cambio de sprite por estado? → Button (Sprite Swap)
- ¿Animaciones complejas? → Button (Animation + Animator)
- ¿Escala/sonido/tooltip simple? → Button + script con interfaces
- ¿Drag & drop/zonas de drop? → Script con `IBeginDrag/IDrag/IEndDrag/IDrop`

5) Interfaces de Unity para UI (EventSystems) — explicación completa

Requisitos para que funcionen:

- Debe existir un `EventSystem` en la escena (Unity lo añade al crear un Button).
- El Canvas debe tener un `GraphicRaycaster` (para UI). En World Space, asigna `worldCamera` al Canvas.
- El objeto que recibe eventos debe estar “raycasteable”: un `Graphic` (Image/Text/TMP) con `raycastTarget = true` o un componente que implemente `ICanvasRaycastFilter`. Si quieres que un GO “vacío” reciba eventos, añade una `Image` transparente y deja `raycastTarget` activado.

Interfaces más usadas y cuándo se disparan:

- `IPointerEnterHandler / IPointerExitHandler`
 - Enter: cuando el puntero entra al rect del objeto; Exit: cuando sale.
 - Uso: resaltar (hover), mostrar/ocultar tooltips.
- `IPointerDownHandler / IPointerUpHandler`

- Down: al presionar el botón del mouse/finger; Up: al soltar.
- Uso: botones que requieren mantener pulsado (cargar ataque), sliders personalizados.
- **IPointerClickHandler**
 - Click se dispara si el Down y el Up ocurren sobre el mismo objeto (con el mismo botón).
 - Uso: acciones de “usar, abrir, confirmar”. Nota: doble click no dispara dos Click por defecto; si quieres doble-click, mídelo con tiempo entre clicks.
- **IBeginDragHandler / IDragHandler / IEndDragHandler**
 - Begin: justo al empezar a arrastrar; Drag: cada frame mientras se arrastra; End: al soltar.
 - Uso: drag & drop de ítems, mover ventanas, reordenar listas.
- **IDropHandler**
 - Se dispara en el objetivo cuando sueltas algo encima (al finalizar un drag).
 - Uso: soltar ítems en slots, equipamiento, contenedores.
- **IScrollHandler**
 - Se dispara al girar la rueda del mouse sobre el elemento.
 - Uso: scroll de listas, zoom.
- **ISelectHandler / IDeselectHandler**
 - Selección de navegación (teclado/gamepad), diferente a hover del mouse.
 - Uso: resaltar foco de navegación, abrir tooltips accesibles por teclado.
- **IMoveHandler**
 - Se dispara con inputs de navegación (flechas/D-Pad) para mover el foco entre elementos UI.
 - Uso: cuadrículas de slots e inventario navegables.
- **ISubmitHandler / ICancelHandler**
 - Submit: activar el elemento (Enter/A botón); Cancel: cancelar/volver (Esc/B botón).
 - Uso: formularios, menús navegables por mando.

Patrones típicos y consejos:

- Tooltips: **IPointerEnter/Exit** para mostrar/ocultar. Añade un pequeño retardo opcional.
- Drag&Drop de inventario: en el ítem/slot, implementa **IBeginDrag/IDrag/IEndDrag**; en el slot destino, **IDropHandler**. Usa un “icono flotante” en un Canvas superior durante el drag y oculta el icono original.
- Hotbar y equipamiento: los receptores (slots) validan el tipo del ítem en **OnDrop** y devuelven feedback si no es válido.
- Evita “event eating”: si un Image hijo captura eventos, quizá el padre no los reciba; desactiva **raycastTarget** del hijo si es solo visual.
- World Space UI: asigna **Canvas.worldCamera** o no recibirás eventos (salvo Overlay). Ajusta orden de sorting si compite con sprites.

Snippet de referencia minimal (hover + click):

```
public class HoverClick : MonoBehaviour, IPointerEnterHandler, IPointerExitHandler,
IPointerClickHandler {
    public UnityEngine.UI.Graphic g; public Color normal=Color.white,
    hover=new(0.9f,0.95f,1,1);
    void Awake(){ if(!g) g = GetComponent<UnityEngine.UI.Graphic>(); if(g) g.color =
    normal; }
```

```

    public void OnPointerEnter(PointerEventData e){ if(g) g.color = hover; }
    public void OnPointerExit(PointerEventData e){ if(g) g.color = normal; }
    public void OnPointerClick(PointerEventData e){ Debug.Log($"Click {e.button}");
}
}

```

Ideas prácticas adicionales:

- Click derecho para menú contextual de ítems (usar/dividir/tirar).
- Reordenar una lista arrastrando sus elementos (Begin/Drag/End + layout dinámico).
- Arrastrar el encabezado de una ventana para moverla (drag sobre el título, no sobre el contenido interactivo).

5.1) Botón “Salir del juego” (Quit) y “Volver al Menú”

Integrado en **GameManager**:

- Botón “Salir”: conecta OnClick → **GameManager.Instance.QuitGame()**
- Botón “Menú”: conecta OnClick → **GameManager.Instance.ExitToMainMenu()**

Consideraciones por plataforma:

- Editor: **QuitGame()** detendrá el Play Mode (no cierra Editor).
- Windows/Mac/Linux/Android: **Application.Quit()** cierra la app (Android puede volver a Home según OEM).
- WebGL: no puede cerrar pestañas; muestra un panel de confirmación y vuelve a la pantalla principal dentro del juego.

6) Inventario básico

Modelo:

- **InventoryModel** (ScriptableObject) con slots con stack. Crea uno: Right Click en Project → Create → PlataformaGame → Inventory → InventoryModel.
- Define capacidad (p.ej. 12).

Datos de ítems:

- **ItemData** (clase serializable simple). Puedes convertir a ScriptableObject si necesitas catálogos grandes.

UI:

1. Canvas In-Game → Panel Inventario (puede estar oculto y abrirse con una tecla o botón).
2. Dentro, un GameObject con GridLayoutGroup (por ejemplo 4 cols x 3 filas).
3. Crea un prefab **Slot** con:
 - Image de fondo (para hover),
 - Image de icono (hijo),
 - Text (cantidad) (u TextMeshProUGUI si usas TMP).

4. Añade **InventoryUI** al Panel, asigna:
- model: tu **InventoryModel** creado,
 - gridRoot: el contenedor con GridLayout,
 - slotPrefab: tu prefab Slot.

Poblar/Probar:

- Desde cualquier script de juego, llama **model.Add(itemData, cantidad)** para ver aparecer los ítems en la UI.
- **InventoryUI** re-renderiza cuando el modelo emite **OnChanged**.

7) Variaciones útiles

- Mostrar/ocultar el inventario con una tecla: activa/desactiva el Panel y pausa o no el juego según prefieras.
- Usar TextMeshPro: cambia los componentes **Text** por **TextMeshProUGUI** y ajusta **InventoryUI** para encontrarlos (o referencia explícita por script).
- Drag & drop: añade interfaces **IBeginDragHandler**, **IDragHandler**, **IEndDragHandler** a **InventoryUISlot** y una capa visual para el icono arrastrado.

8) Consejos de organización

- Capas: Menús (Screen Space Overlay), HUD (Overlay), Barras de vida de enemigos (World Space) para claridad.
- Sonidos de UI: añade **AudioSource** y dispara clips en hover/click desde **UIButtonHover**.
- Navegación por gamepad/teclado: usa Navigation de los Button si lo necesitas.

9) Integración con GameManager (unificado)

- **GameManager** ahora maneja Pause/Resume/Reset, metas y win panel.
- El HUD expone **UI_Resume**, **UI_RestartLevel**, **UI_ExitToMainMenu** para conectar botones.
- HUD escucha eventos (pausa/resume/level reset/coin/enemy) y actualiza textos.

10) Checklist rápido

- ☐ Scenes en Build Settings (MainMenu, Level_01)
- ☐ Canvas MainMenu con botones y **MainMenuController**
- ☐ Canvas Pausa con **PauseMenuController** y **PauseInput** en escena
- ☐ InventoryModel asset, **InventoryUI** configurado y slotPrefab hecho
- ☐ EventSystem presente en escenas con UI

Con esto tienes lo esencial: pausa con ESC y hover/click con interfaces, menú principal funcional y un inventario visual básico que puedes expandir.